看门狗(WDT)

看门狗(WDT)
- 例程实验效果简介
  - 1.什么是看门狗?
  - 2.看门狗的工作原理
  - 3. 主要用途
- 例程代码
  - 完整代码
  - 例程代码流程图

例程实验效果简介

1.什么是看门狗?

看门狗(Watchdog Timer)是嵌入式系统中的一种硬件定时器，主要用于系统的自我检测和恢复。它就像一个"监督者"，通过定时器持续监控系统运行状态，当系统出现异常时可以自动触发系统复位。

2.看门狗的工作原理

看门狗的工作原理包含以下几个关键环节：

- 初始化：设置看门狗并指定超时时间
- 运行监控："喂狗"操作定期重置计时器
- 复位触发：超时未"喂狗"则自动重启系统

3. 主要用途

- 检测并防止系统死机
- 提升系统运行可靠性
- 提供自动恢复机制
- 减少人工干预需求

例程代码

本节例程代码位于【源码汇总 / 02.Basic / 08.wdt.py 】

完整代码

- 注：以【源码汇总 / 02.Basic / 08.wdt.py 】文件内容为准

```
# 导入必要的模块 | Import required modules
from machine import WDT  # 导入看门狗模块 | Import watchdog module
import time  # 导入时间模块 | Import time module

"""
详细解析 | Detailed Analysis:
WDT (Watchdog Timer) 是一种硬件定时器，如果在规定时间内没有被复位（喂狗），
系统就会自动重启。主要用于检测和恢复系统故障。

WDT is a hardware timer that will automatically restart the system if not reset
```

```
("fed") within a specified time. It's mainly used for detecting and recovering
from system failures.
"""

def init_watchdog(id=1, timeout=3):
    """初始化看门狗 | Initialize watchdog
    Args:
        id: 看门狗ID | Watchdog ID
        timeout: 超时时间(秒) | Timeout period(seconds)
    Returns:
        WDT对象 | WDT object

    详细解析 | Analysis:
    创建WDT对象，设置超时时间。如果超过timeout秒没有喂狗，系统将重启。
    Creates WDT object and sets timeout. System will restart if not fed within
timeout seconds.
    """
    try:
        return WDT(id, timeout)
    except Exception as e:
        print(f"看门狗初始化失败 | Watchdog initialization failed: {e}")
        return None

def feed_watchdog(wdt, feed_times=3, interval=1):
    """定时喂狗 | Feed watchdog periodically
    Args:
        wdt: 看门狗对象 | Watchdog object
        feed_times: 喂狗次数 | Number of feeds
        interval: 喂狗间隔(秒) | Interval between feeds(seconds)

    详细解析 | Analysis:
    按照指定间隔喂狗指定次数。这个过程会持续 feed_times * interval 秒。
    Feeds the watchdog specified times at given intervals. This process will
last
    for feed_times * interval seconds.
    """
    if not wdt:
        return

    try:
        for i in range(feed_times):
            time.sleep(interval)  # 等待指定时间 | Wait for specified interval
            wdt.feed()  # 喂狗操作 | Feed the watchdog
            print(f"第{i+1}次喂狗 | Feed watchdog {i+1} times")
    except Exception as e:
        print(f"喂狗过程出错 | Error during feeding: {e}")

def main():
    """主函数 | Main function

    详细解析 | Analysis:
    程序的主要执行流程:
    1. 初始化看门狗
    2. 执行定时喂狗
    3. 停止喂狗等待系统重启

    Main program flow:
    1. Initialize watchdog
```

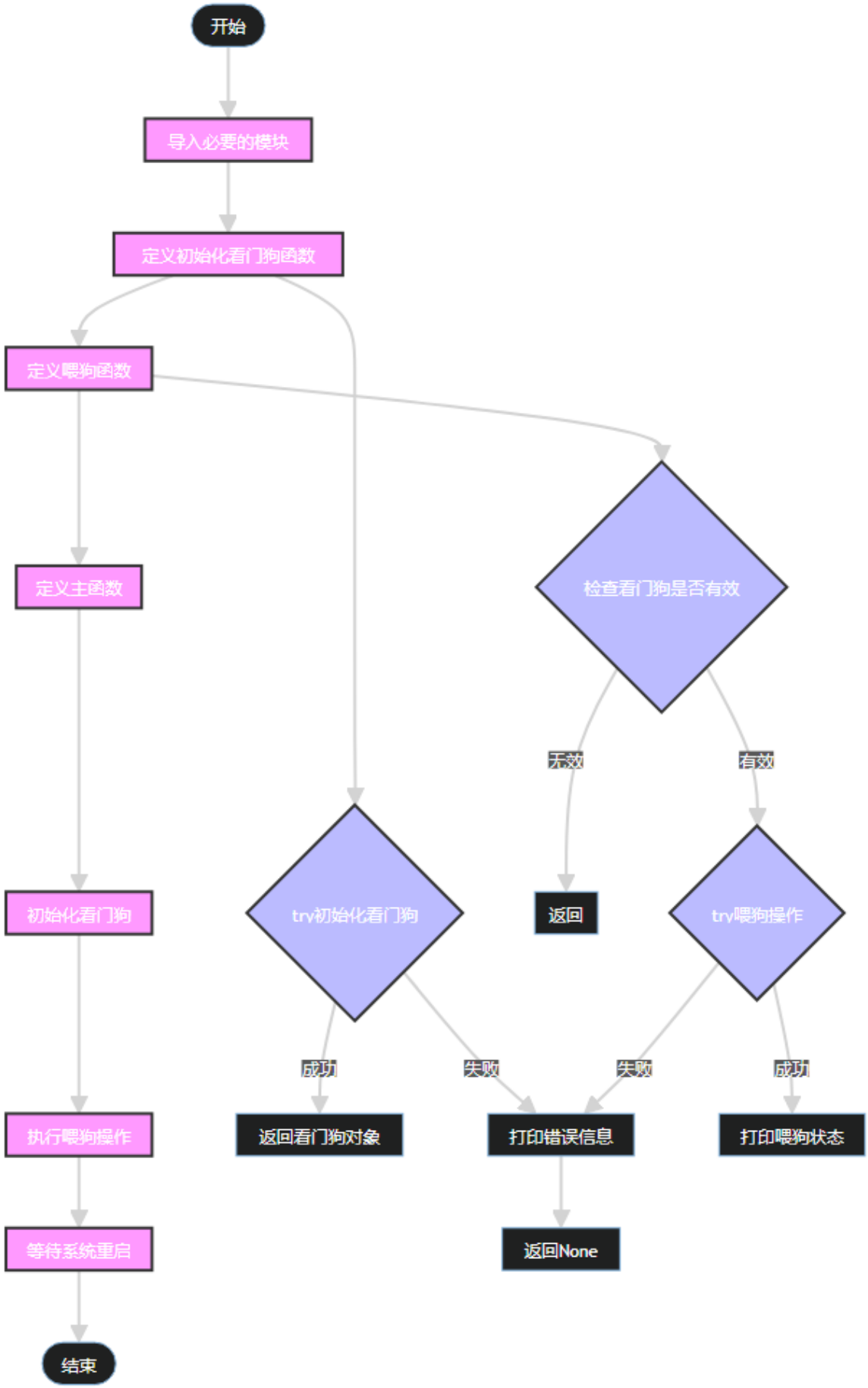
```
    2. Perform periodic feeding
    3. Stop feeding and wait for system restart
    """
    # 初始化看门狗 | Initialize watchdog
    wdt = init_watchdog()
    if not wdt:
        return

    # 喂狗循环 | Feeding loop
    feed_watchdog(wdt)

    # 等待重启 | Wait for restart
    print("停止喂狗，等待系统重启... | Stop feeding, waiting for system restart...")
    while True:
        time.sleep(0.1)  # 降低CPU占用 | Reduce CPU usage

if __name__ == '__main__':
    main()
```

例程代码流程图


开始
导入必要的模块
定义初始化看门狗函数
定义喂狗函数
定义主函数
初始化看门狗
执行喂狗操作
等待系统重启
结束
try初始化看门狗
成功
失败
返回看门狗对象
检查看门狗是否有效
无效
返回
有效
try喂狗操作
失败
打印错误信息
返回None
成功
打印喂狗状态